

DOCUMENTACIÓN

Diagrama de Clases Sistema de Gestión de Inventario

1. Introducción

Este documento describe en detalle la arquitectura del sistema de gestión de inventario mediante el análisis de su Diagrama de Clases. El diagrama recoge las entidades principales, sus atributos, métodos y las relaciones entre ellas.

La aplicación está desarrollada en Java con interfaz gráfica Swing y sigue el patrón de diseño DAO (Data Access Object) para separar la lógica de negocio del acceso a la base de datos.

2. Visión general de la arquitectura

El sistema se organiza en cuatro capas principales:

- Capa de Modelo: clases de dominio (Usuario, Material, Roles).
- Capa de Acceso a Datos: clases DAO (UsuarioDAO, MaterialDAO) y AccesoBaseDatos.
- Capa de Presentación: ventanas y diálogos Swing (LoginFrame, AdminPanelFrame, ProfesorPanelFrame y diálogos modales).
- Utilidades: Validador y GestorCSV.

2.1 Roles de usuario

El sistema define dos roles de acceso mediante la enumeración Roles:

- ADMINISTRADOR — acceso completo: alta, edición, baja de materiales, gestión de usuarios e importación/exportación CSV.
- PROFESOR — acceso de solo lectura: consulta y filtrado del inventario.

2.2 Relaciones entre clases

Clase origen	Clase destino	Tipo de relación
LoginFrame	UsuarioDAO	Dependencia (usa para autenticación)
LoginFrame	AdminPanelFrame / ProfesorPanelFrame	Dependencia (navega según rol)
AdminPanelFrame	MaterialDAO	Dependencia (consultas de inventario)
AdminPanelFrame	AltaMaterialDialog / EditarMaterialDialog	Agregación (abre diálogos modales)
AdminPanelFrame	UsuariosDialog / ImportarExportarDialog / AnadirProfeDialog	Agregación (abre diálogos modales)
ProfesorPanelFrame	MaterialDAO	Dependencia (consultas de solo lectura)
UsuarioDAO / MaterialDAO	AccesoBaseDatos	Dependencia (obtiene Connection)
UsuarioDAO	Usuario	Crea / devuelve instancias
MaterialDAO	Material	Crea / devuelve instancias
UsuarioDAO	Roles	Usa la enumeración para asignar rol

Clase origen	Clase destino	Tipo de relación
AltaMaterialDialog / EditarMaterialDialog	MaterialDAO	Dependencia (persistencia)
AltaMaterialDialog / EditarMaterialDialog	Validador	Dependencia (validación de campos)
ImportarExportarDialog	GestorCSV	Dependencia (importar/exportar)

3. Descripción detallada de las clases

3.1 Usuario

Representa a un usuario del sistema con sus credenciales y rol asignado.

Visibilidad	Nombre / Firma	Descripción
ATRIBUTOS		
- (privado)	id: int	Identificador único del usuario
- (privado)	nombre: String	Nombre de usuario para el login
- (privado)	contrasena: String	Contraseña del usuario
- (privado)	rol: Enum	Rol del usuario (PROFESOR o ADMINISTRADOR)
MÉTODOS		
+ (público)	getId(): int	Retorna el identificador del usuario
+ (público)	getNombre(): String	Retorna el nombre del usuario
+ (público)	getContrasena(): String	Retorna la contraseña del usuario
+ (público)	getRol(): Enum	Retorna el rol del usuario
+ (público)	toString(): String	Representación textual del objeto

3.2 Roles

Enumeración que define los roles disponibles en el sistema.

Visibilidad	Nombre / Firma	Descripción
ATRIBUTOS		

Visibilidad	Nombre / Firma	Descripción
(constante)	PROFESOR	Rol con permisos de consulta y filtrado de inventario
(constante)	ADMINISTRADOR	Rol con permisos completos de gestión

3.3 Material

Representa un material o recurso del inventario con todos sus atributos descriptivos y de catalogación.

Visibilidad	Nombre / Firma	Descripción
ATRIBUTOS		
- (privado)	id: int	Identificador único del material
- (privado)	nombre: String	Nombre del material
- (privado)	descripcion: String	Descripción detallada del material
- (privado)	id_categoria: int	Referencia a la categoría del material
- (privado)	id_subcategoria: int	Referencia a la subcategoría del material
- (privado)	id_estado: int	Estado actual del material
- (privado)	cantidad: int	Cantidad disponible en inventario
- (privado)	id_ubicacion: int	Referencia a la ubicación física del material
- (privado)	fecha_alta: LocalDate	Fecha de registro en el sistema
- (privado)	observaciones: String	Observaciones adicionales
MÉTODOS		
+ (público)	getId(): int	Retorna el identificador del material
+ (público)	getNombre(): String	Retorna el nombre
+ (público)	getDescripcion(): String	Retorna la descripción
+ (público)	getId_categoria(): int	Retorna el ID de categoría
+ (público)	getId_subcategoria(): int	Retorna el ID de subcategoría
+ (público)	getId_estado(): int	Retorna el ID de estado

Visibilidad	Nombre / Firma	Descripción
+ (público)	getCantidad(): int	Retorna la cantidad disponible
+ (público)	getId_ubicacion(): int	Retorna el ID de ubicación
+ (público)	getFecha_alta(): LocalDate	Retorna la fecha de alta
+ (público)	getObservaciones(): String	Retorna las observaciones

3.4 AccesoBaseDatos

Gestiona la configuración y apertura de la conexión a la base de datos.

Visibilidad	Nombre / Firma	Descripción
ATRIBUTOS		
- (privado)	bd: String	Nombre de la base de datos
- (privado)	url: String	URL de conexión JDBC
- (privado)	usuario: String	Usuario de la base de datos
- (privado)	clave: String	Contraseña de la base de datos
- (privado)	conn: Connection	Objeto de conexión activo
MÉTODOS		
+ (público)	cargarConfiguracion(): void	Carga los parámetros desde el archivo de configuración
+ (público)	abrirConexion(): void	Abre la conexión a la base de datos
+ (público)	getConn(): Connection	Retorna el objeto Connection activo
+ (público)	cerrar(): boolean	Cierra la conexión y retorna el resultado

3.5 UsuarioDAO

Objeto de Acceso a Datos para la entidad Usuario. Gestiona todas las operaciones CRUD sobre la tabla de usuarios.

Visibilidad	Nombre / Firma	Descripción
MÉTODOS		
- (privado)	getConexion(): Connection	Obtiene la conexión a la base de datos
- (privado)	crearUsuario(ResultSet rs): Usuario	Construye un objeto Usuario desde un ResultSet

Visibilidad	Nombre / Firma	Descripción
+ (público)	anadirProfe(Usuario u): void	Añade un nuevo profesor al sistema
+ (público)	existeUsuario(String nombre): boolean	Comprueba si ya existe un usuario con ese nombre
+ (público)	eliminarProfe(String nombre): boolean	Elimina el profesor con el nombre indicado
+ (público)	verUsuarios(): List<Usuario>	Retorna la lista completa de usuarios
+ (público)	login(String nombre, String contrasena): Usuario	Valida credenciales y retorna el Usuario autenticado o null

3.6 MaterialDAO

Objeto de Acceso a Datos para la entidad Material. Gestiona todas las operaciones sobre el inventario de materiales.

Visibilidad	Nombre / Firma	Descripción
MÉTODOS		
- (privado)	getConexion(): Connection	Obtiene la conexión a la base de datos
- (privado)	crearMaterial(ResultSet): Material	Construye un objeto Material desde un ResultSet
+ (público)	actualizarMaterial(int cantidad, int idEstado, String obser, String nombre): boolean	Actualiza los datos de un material existente
+ (público)	insertarMaterial(Material m): void	Inserta un nuevo material en la base de datos
+ (público)	verInventario(): List<Material>	Retorna todos los materiales del inventario
+ (público)	filtrarPorNombre(String nombre): List<Material>	Filtra materiales por nombre
+ (público)	filtrarPorCategoria(String categoria): List<Material>	Filtra materiales por categoría
+ (público)	filtrarPorEstado(String estado): List<Material>	Filtra materiales por estado
+ (público)	filtrarPorCodigo(int id): List<Material>	Filtra materiales por código ID
+ (público)	getIdCategoria(String nombre): int	Obtiene el ID de una categoría por nombre
+ (público)	getIdSubcategoria(String nombre): int	Obtiene el ID de una subcategoría por nombre
+ (público)	getIdEstado(String nombre): int	Obtiene el ID de un estado por nombre

Visibilidad	Nombre / Firma	Descripción
+ (público)	getIdUbicacion(String espacio): int	Obtiene el ID de una ubicación por nombre

3.7 Validador

Proporciona métodos estáticos de validación de datos de entrada antes de ser procesados.

Visibilidad	Nombre / Firma	Descripción
MÉTODOS		
+ (público)	leerNombre(String nombre): boolean	Valida que el nombre no esté vacío ni sea nulo
+ (público)	leerContrasena(String contrasena): boolean	Valida que la contraseña cumpla los requisitos mínimos
+ (público)	leerMaterial(String nombre): boolean	Valida el nombre de un material
+ (público)	leerDescripcion(String descripcion): boolean	Valida la descripción de un material
+ (público)	leerCantidad(int cantidad): boolean	Valida que la cantidad sea un valor positivo

3.8 GestorCSV

Gestiona la importación y exportación de datos del inventario en formato CSV.

Visibilidad	Nombre / Firma	Descripción
MÉTODOS		
+ (público)	exportar(String ruta): void	Exporta el inventario a un archivo CSV en la ruta indicada
+ (público)	importar(String ruta): int	Importa materiales desde un CSV y retorna el número de registros importados

3.9 LoginFrame

Interfaz gráfica de inicio de sesión. Primer formulario que ve el usuario al arrancar la aplicación.

Visibilidad	Nombre / Firma	Descripción
MÉTODOS		

Visibilidad	Nombre / Firma	Descripción
- (privado)	initComponents(): void	Inicializa y coloca los componentes Swing de la ventana
+ (público)	btnEntrarActionPerformed(ActionEvent evt): void	Manejador del botón Entrar: valida credenciales y redirige según rol
+ (público)	txtContrasenaActionPerformed(ActionEvent evt): void	Manejador de la tecla Enter en el campo contraseña

3.10 AdminPanelFrame

Panel principal del administrador. Permite la gestión completa del inventario y de los usuarios del sistema.

Visibilidad	Nombre / Firma	Descripción
MÉTODOS		
- (privado)	cargarTabla(): void	Carga los materiales en la tabla de la interfaz
+ (público)	btnCerrarSesionActionPerformed(ActionEvent evt): void	Cierra la sesión y vuelve al LoginFrame
+ (público)	txtBuscarActionPerformes(ActionEvent evt): void	Filtra la tabla al escribir en el campo de búsqueda
+ (público)	btnAltaActionPerformed(ActionEvent evt): void	Abre el diálogo de alta de nuevo material
+ (público)	btnEditarActionPerformed(ActionEvent evt): void	Abre el diálogo de edición del material seleccionado
+ (público)	btnDarDeBajaActionPerformed(ActionEvent evt): void	Da de baja el material seleccionado
+ (público)	btnLocalizarActionPerformed(ActionEvent evt): void	Muestra la ubicación del material seleccionado
+ (público)	cmbEstadoActionPerformed(ActionEvent evt): void	Filtra por estado al cambiar el combo
+ (público)	cmbCategoriaActionPerformed(ActionEvent evt): void	Filtra por categoría al cambiar el combo
+ (público)	btnUsuariosActionPerformed(ActionEvent evt): void	Abre el diálogo de gestión de usuarios
+ (público)	btnInventarioActionPerformed(ActionEvent evt): void	Recarga el inventario completo
+ (público)	btnImporExporActionPerformed(ActionEvent evt): void	Abre el diálogo de importar/exportar CSV

3.11 ProfesorPanelFrame

Panel principal del profesor. Permite consultar y filtrar el inventario sin poder modificarlo.

Visibilidad	Nombre / Firma	Descripción
MÉTODOS		
- (privado)	cargarTabla(): void	Carga el inventario en la tabla
+ (público)	btnCerrarSesionActionPerformed(ActionEvent evt): void	Cierra la sesión del profesor
+ (público)	txtBuscarActionPerformed(ActionEvent evt): void	Filtra la tabla por texto de búsqueda
+ (público)	btnLocalizarActionPerformed(ActionEvent evt): void	Muestra la ubicación del material seleccionado
+ (público)	cmbEstadoActionPerformed(ActionEvent evt): void	Filtra por estado
+ (público)	cmbCategoriaActionPerformed(ActionEvent evt): void	Filtra por categoría

3.12 AltaMaterialDialog

Diálogo modal para dar de alta un nuevo material en el inventario.

Visibilidad	Nombre / Firma	Descripción
MÉTODOS		
+ (público)	btnGuardarActionPerformed(ActionEvent evt): void	Valida y guarda el nuevo material en la base de datos
+ (público)	btnCancelarActionPerformed(ActionEvent evt): void	Cancela la operación y cierra el diálogo

3.13 EditarMaterialDialog

Diálogo modal para editar los datos de un material existente.

Visibilidad	Nombre / Firma	Descripción
MÉTODOS		
+ (público)	btnGuardarActionPerformed(ActionEvent evt): void	Guarda los cambios del material editado
+ (público)	btnCerrarActionPerformed(ActionEvent evt): void	Cierra el diálogo sin guardar cambios

3.14 AnadirProfeDialog

Diálogo modal para añadir un nuevo profesor al sistema.

Visibilidad	Nombre / Firma	Descripción
MÉTODOS		
+ (público)	btnGuardarActionPerformed(ActionEvent evt): void	Valida y guarda el nuevo profesor
+ (público)	btnCancelarActionPerformed(ActionEvent evt): void	Cancela la operación y cierra el diálogo

3.15 UsuariosDialog

Diálogo de gestión de usuarios, accesible únicamente desde el panel de administrador.

Visibilidad	Nombre / Firma	Descripción
MÉTODOS		
+ (público)	btnAnadirActionPerformed(ActionEvent evt): void	Abre el diálogo para añadir un nuevo profesor
+ (público)	btnActualizarActionPerformed(ActionEvent evt): void	Actualiza la lista de usuarios
+ (público)	btnEliminarActionPerformed(ActionEvent evt): void	Elimina el usuario seleccionado
- (privado)	cargarTabla(): void	Carga los usuarios en la tabla
+ (público)	btnCerrarPerformed(ActionEvent evt): void	Cierra el diálogo

3.16 ImportarExportarDialog

Diálogo para importar o exportar el inventario en formato CSV.

Visibilidad	Nombre / Firma	Descripción
MÉTODOS		
+ (público)	btnExportarActionPerformed(ActionEvent evt): void	Lanza la exportación del inventario a CSV
+ (público)	btnImportarActionPerformed(ActionEvent evt): void	Lanza la importación de materiales desde CSV
+ (público)	btnCerrarActionPerformed(ActionEvent evt): void	Cierra el diálogo

4. Convenciones de nomenclatura y visibilidad

El diagrama sigue la notación UML estándar para indicar la visibilidad de atributos y métodos:

Símbolo	Significado
+	Público — accesible desde cualquier clase
-	Privado — accesible solo desde la propia clase
#	Protegido — accesible desde la clase y sus subclases

5. Patrones de diseño aplicados

5.1 Data Access Object (DAO)

Las clases UsuarioDAO y MaterialDAO implementan el patrón DAO, aislando la lógica de acceso a la base de datos del resto de la aplicación. Esto facilita el mantenimiento y la posibilidad de cambiar el motor de base de datos sin modificar la lógica de negocio.

5.2 MVC simplificado

La arquitectura sigue un modelo simplificado de MVC (Model-View-Controller) donde las clases de dominio (Usuario, Material) actúan como Modelo, los Frame y Dialog constituyen la Vista, y los DAO ejercen de Controlador al mediar entre ambas capas.

5.3 Singleton implícito en AccesoBaseDatos

La clase AccesoBaseDatos centraliza la gestión de la conexión a la base de datos, siendo utilizada por los DAO para obtener siempre la misma instancia de Connection y evitar múltiples conexiones simultáneas innecesarias.

6. Resumen de clases

Clase	Tipo	Responsabilidad principal
Usuario	Entidad	Modelo de datos del usuario del sistema
Roles	Enumeración	Define los roles PROFESOR y ADMINISTRADOR
Material	Entidad	Modelo de datos del material del inventario
AccesoBaseDatos	Utilidad	Gestión de la conexión a la base de datos
UsuarioDAO	DAO	CRUD de usuarios sobre la base de datos
MaterialDAO	DAO	CRUD de materiales sobre la base de datos
Validador	Utilidad	Validación de entradas de usuario
GestorCSV	Utilidad	Importación y exportación CSV del inventario
LoginFrame	Vista	Pantalla de inicio de sesión
AdminPanelFrame	Vista	Panel de control del administrador

Clase	Tipo	Responsabilidad principal
ProfesorPanelFrame	Vista	Panel de consulta del profesor
AltaMaterialDialog	Vista	Diálogo para añadir un nuevo material
EditarMaterialDialog	Vista	Diálogo para editar un material existente
AnadirProfeDialog	Vista	Diálogo para añadir un nuevo profesor
UsuariosDialog	Vista	Diálogo de gestión de usuarios
ImportarExportarDialog	Vista	Diálogo de importación/exportación CSV